

EE 2361 Final Project: AirDraw

Abdifatah Hassan, Ali Endris, Idres Ali, Adillo Sero

Introduction:

The AirDraw API is a PIC24 library that combines physical kinetic movement and pixel-based art. By abstracting the complex hardware layers of the PIC24 architecture, the library allows developers to focus on creative logic rather than peripheral management, allowing for nanosecond-level timing using the WS2812B (NeoPixel) LEDs through cycle-accurate assembly bit-banging to ensure stable visual rendering, while simultaneously providing a streamlined interface for the MPU-6050 IMU that handles I2C bus transactions and applies a built-in Alpha Filter to transform shaky, high-resolution physical signals into fluid cursor movements on an 8 by 8 digital canvas. Furthermore, the library features a sophisticated Timestamp-based Click Engine that uses the system's global timer to differentiate between single, double, and triple button presses, providing a robust and modular foundation for any real-time, motion-driven interaction.

Hardware Description:

Component	Specific Part	Description	Link
Joystick	Adafruit 512	2 Axis Potentiometer Joystick with Select button	https://www.adafruit.com/product/512?srltid=AfmBOoocNM2duaVdg-6pE7N842s4HtOzxoCvaJhmyFCTUCP3zeywMgBr
LED Matrix	WS2812B 8x8	64-pixel RGB array	https://www.adafruit.com/product/1487?gad_source=1&gad_campaignid=21079227318&gclid=CjwKCAjwzevPBhBaEiwAplAxvsiFhpEyx9UBT0ZyS3N4udMJHfKdRT-AG3bdAl8953okbeJXgDjaLhoChXsQAvD_BwE
IMU Sensor	MPU-6050	3D motion tracking measurement tool	https://www.adafruit.com/product/3886?srltid=AfmBOordTywSE4gWAMbefECHbMRNYu

			DtGtNhkKsaHT-vNYhDYQe22pUh
--	--	--	--

Full Documentation:

void **init_hardware**(void):

Description: Configures the PIC24, initializes ADC for joystick pins, sets up Timer2 for 1-second interval tracking, and wakes the IMU from sleep mode.

Inputs: N/A

Outputs: N/A

void **get_air_coords**(int *x, int *y):

Description: Handles motion control. It reads how much the device is tilting, smooths out any shaky hand movements so the cursor doesn't flicker, and converts that data into simple 0–7 coordinates that fit on the LED screen.

Inputs: Pointers to integers x and y.

Outputs: N/A

uint16_t **read_adc**(uint8_t channel):

Description: Checks the physical position of the joystick. It looks at a specific direction (left/right or up/down) and turns that physical position into a number between 0 and 1023 so the code can use it.

Inputs: uint8_t channel

Outputs: uint16_t (0-1023)

Action **check_button_actions**(void):

Description: Analyzes button press timing using the system's global “timeCounter” and checks your presses to see if you are clicking once, double-clicking, or triple-clicking to change modes.

Inputs: N/A

Outputs: Action (NO_CLICK, SINGLE_CLICK, DOUBLE_CLICK, TRIPLE_CLICK)

void **update_matrix**(int curX, int curY):

Description: Draws everything onto the LED matrix. It sends your saved artwork to the lights and adds a faint "cursor" dot at your current position so you know where you are pointing.

Inputs: Position to place cursor (X & Y)

Outputs: N/A

Basic Usage Example:

The basic usage of this library establishes a direct link between the physical movement of the device and the LED matrix. Upon running, the code initializes the system and enters a continuous loop that polls the MPU-6050 sensor for motion data. The library automatically smooths this data using an Alpha Filter to prevent the cursor from jittering, translates the raw tilt into a simple 0–7 coordinate grid, and updates the display. The result is a single, steady LED cursor that follows the tilt of the device in real-time.

Advanced Usage Example:

This example however demonstrates mode swapping via triple-click, drawing through button holding, and coordinate filtering.

The advanced usage case transforms the hardware into a multi-functional digital sketchbook by utilizing the library's full suite of input processing. In this scenario, the code runs a state-driven loop that uses a triple-click to swap between manual joystick control and motion-based "air-drawing," clearing the screen automatically during each transition. The system continuously tracks the cursor position using either raw ADC sampling or filtered IMU data, while simultaneously monitoring the button's physical state; if held down, the library writes color data to the persistent pixelBuffer, effectively painting onto the matrix. This implementation demonstrates how the library manages complex user interactions, smoothed sensor fusion, and canvas memory all at once.